AdapterNAS: Training-Free Neural Architecture Search for Foundation Model Adapter Topologies

Aayam Bansal Ishaan Gangwani
Sixth Workshop on Neural Architecture Search (CVPR-NAS26)

Abstract

Parameter-efficient fine-tuning (PEFT) methods like LoRA adapt foundation models by inserting low-rank modules, yet the adapter topology*—which layers, modules, and ranks to use—is typically chosen by hand. We propose AdapterNAS, a training-free NAS framework that searches this adapter topology using zero-cost proxies. Given a pre-trained Vision Transformer, AdapterNAS scores candidate LoRA configurations via an ensemble of GradNorm, SNIP, Fisher, and entropy proxies on a single calibration batch, then refines the search space with proxy-guided evolutionary optimization. On CIFAR-100 with only 2% training data and ViT-B/16, AdapterNAS searches 175 configurations in under 15 minutes on a single GPU, with the proxy-selected top-5 always containing the oracle-best configuration (79.3% accuracy, vs. 60.0% linear probe). GradNorm alone achieves Spearman $\rho = 0.62$ with downstream accuracy ($p < 0.001$). We validate across backbones (ViT-S/16), data regimes (1%–10%, up to 89.0% at 10% data), and datasets (Flowers-102, where all configs achieve >99%), showing consistent proxy-guided selection quality. Our analysis reveals that adapter topology significantly impacts performance—a 19 percentage point spread across configurations—and that MLP adaptation, typically ignored, contributes as much as attention adaptation.*

1. Introduction

Low-Rank Adaptation (LoRA) [8] and related PEFT methods [7, 9] have become the standard for adapting foundation models. Practitioners must choose *where* to insert adapters (which layers, which modules) and *how large* they should be (rank per module). These decisions profoundly impact performance—as we show, the accuracy range across adapter topologies exceeds 19 percentage points on CIFAR-100—yet are typically made via rules of thumb or expensive grid search.

Neural Architecture Search (NAS) offers principled tools for such design decisions. In particular, training-free NAS uses zero-cost proxies [1, 15] to score architectures without any gradient steps. However, existing training-free NAS targets *backbone* architectures, not the adapter topology *within* a frozen pre-trained model.

We bridge this gap with AdapterNAS, which formulates adapter topology selection as a NAS problem and solves it with zero-cost proxies. Our contributions:

1. **Formulation**: We define a 5^{36}-configuration search space over LoRA adapter topologies in ViT and propose proxy-guided evolutionary search (Algorithm 1) to efficiently navigate it.
2. **Strong proxy signal**: GradNorm achieves $\rho = 0.62$ Spearman correlation with downstream accuracy on CIFAR-100 (40 evaluated configs, $p < 0.001$), and proxy top-5 selection finds the oracle-best configuration with zero regret.
3. **Comprehensive evaluation**: We report results across two backbones (ViT-B/16, ViT-S/16), four data regimes (1%–10%), and two datasets, with full Pareto analysis and latency benchmarks.
4. **Design insights**: MLP adaptation is undervalued; uniform moderate-rank adapters outperform layer-selective strategies; early layers benefit more from adaptation than late layers.

2. Related Work

Parameter-Efficient Fine-Tuning. LoRA [8] injects trainable low-rank matrices alongside frozen weights. QLoRA [4] extends this to quantized models. Vision-specific methods include AdaptFormer [2], SSF [13], and VPT [9]. He *et al*. [6] unify adapters, prefix tuning, and LoRA. VeRA [10] and DoRA [14] further improve parameter efficiency. Despite this rich landscape, the *topology*—where to place adapters and at what capacity—remains manually specified.

Adaptive Rank Allocation. AdaLoRA [22] prunes singular values during training to dynamically allocate rank. DyLoRA [18] trains across rank ranges simultaneously. AutoPEFT [23] searches over PEFT module combinations. These methods improve over fixed configurations but re-

quire *training* to evaluate candidates, making large-scale exploration expensive.

Training-Free NAS. Zero-cost proxies evaluate architectures without training using gradient statistics [1], pruning criteria [12, 17, 19], or information-theoretic measures [3, 15]. These methods have been benchmarked on NAS-Bench [5, 21] but applied only to backbone search, not adapter topologies. Our work is, to our knowledge, the first to apply training-free NAS *inside* the adapter configuration space of pre-trained models.

3. Method

3.1. Problem Formulation

Given a pre-trained ViT with $L = 12$ blocks, each containing a combined QKV attention projection and an MLP with fc1/fc2 layers, we define an adapter configuration $\mathcal{A}$:

$$\mathcal{A} : \{1, \ldots, L\} \times \{\text{qkv}, \text{fc1}, \text{fc2}\} \to \{0, 4, 8, 16, 32\} \quad (1)$$

where $\mathcal{A}(\ell, m) = r$ assigns rank r to layer ℓ, module m ($r = 0$ means no adapter). This yields $5^{36} \approx 1.5 \times 10^{25}$ possible configurations.

For each adapter, we apply LoRA: $W' = W + BA$ with $A \in \mathbb{R}^{d_{\text{in}} \times r}$, $B \in \mathbb{R}^{r \times d_{\text{out}}}$, where only A, B are trainable.

3.2. Zero-Cost Proxy Ensemble

Each candidate $\mathcal{A}$ is scored using four proxies computed on a calibration batch $\mathcal{D}_{\text{cal}}$ of 128 samples *without any training*:

GradNorm: $\text{GN}(\mathcal{A}) = \sum_{p \in \Theta_{\mathcal{A}}} \|\nabla_p \mathcal{L}\|_2$ — the sum of gradient L_2 norms, measuring initial training signal.

SNIP [12]: $\text{SNIP}(\mathcal{A}) = \sum_p |\nabla_p \mathcal{L} \cdot p|$ — connection sensitivity.

Fisher: $\text{Fisher}(\mathcal{A}) = \sum_p (\nabla_p \mathcal{L})^2$ — diagonal Fisher information.

NegEntropy: $\text{NE}(\mathcal{A}) = -\mathbb{E}_x[H(f_{\theta,\mathcal{A}}(x))]$ — prediction confidence (label-free).

Each proxy is min-max normalized across all evaluated candidates, and the combined score averages the four normalized proxies. As we show in Sec. 4.3, GradNorm alone is the strongest individual proxy ($\rho = 0.62$), while the combined ensemble achieves $\rho = 0.32$.

3.3. Proxy-Guided Evolutionary Search

Rather than exhaustively scoring random configurations, we use an evolutionary strategy guided by the proxy ensemble. Algorithm 1 describes the procedure.

The search scores 175 total configurations: 25 structured + 30 random + 120 evolutionary (6 generations $\times$ 20 per generation). This completes in $\sim$15 minutes on a single A10G GPU, requiring only forward-backward passes on a 128-sample calibration batch per configuration.

Algorithm 1 Proxy-guided evolutionary adapter search

Require: Pre-trained model f_θ, calibration data $\mathcal{D}_{\text{cal}}$, population size N=20, generations G=6, mutation rate μ=0.3
Score N_{init}=55 initial configs (25 structured + 30 random) with proxy ensemble
$P_0 \leftarrow$ top-N initial configs by combined proxy score
for $g = 1$ to G **do**
  Selection: keep top-$N/2$ parents from P_{g-1}
  Crossover: for each pair of parents, produce 1 offspring via uniform crossover over the 36 (layer, module) entries
  Mutation: for each offspring, with probability μ per entry, resample rank from $\{0, 4, 8, 16, 32\}$
  Score all offspring with proxy ensemble
  $P_g \leftarrow$ top-N from parents $\cup$ offspring
end for
return All scored configurations (initial + evolutionary)

4. Experiments

4.1. Setup

Model. ViT-B/16 (86M params, 12 blocks, d=768, d_{mlp}=3072) pretrained on ImageNet-21k via `timm` [20]. We also evaluate ViT-S/16 (22M params, 12 blocks, d=384) for cross-backbone analysis.

Datasets. (1) **CIFAR-100** [11]: 100 classes. We use 2% of training data (1,000 samples) for fine-tuning and 2,000 test samples for validation. (2) **Flowers-102** [16]: 102 categories, using combined train+val (2,040 images) for training and the 6,149 test images for evaluation.

LoRA implementation. We wrap each target module (QKV, fc1, fc2) with a custom LoRA layer, since timm's combined QKV projection is incompatible with standard PEFT libraries. The classification head is always trainable.

Training. AdamW, lr $= 10^{-3}$, weight decay 0.01, cosine schedule, gradient clipping at 1.0, 5 epochs on CIFAR-100 and 15 epochs on Flowers-102.

Latency. Measured with CUDA synchronization: 20 warmup + 50 timed runs per configuration.

4.2. Main Results: CIFAR-100

Table 1 shows the main CIFAR-100 results. The best configuration (rand_2) achieves **79.3%**, a 19.3 percentage point improvement over linear probing and 3.6 points above uniform r=8. Key observations:

Topology matters profoundly. Across 40 fine-tuned configurations, accuracy ranges from 60.0% (linear probe) to

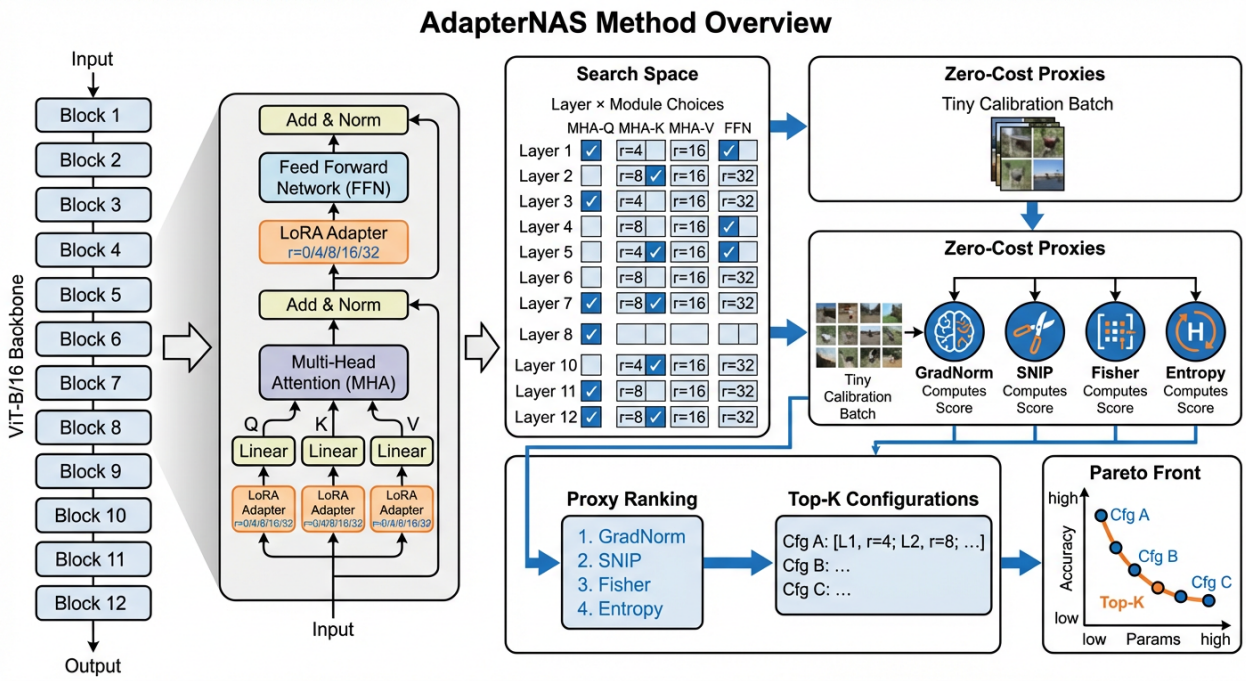

Figure 1. **Overview of AdapterNAS.** Given a pre-trained ViT, we define a search space over adapter topologies: for each transformer block and module (QKV, MLP-fc1, MLP-fc2), we select a LoRA rank from $\{0, 4, 8, 16, 32\}$. Phase 1: structured and random configurations are proxy-scored. Phase 2: evolutionary search refines the space using proxy-guided selection. Phase 3: top-ranked candidates are validated via short fine-tuning.

Table 1. **CIFAR-100 results (ViT-B/16, 2% data).** 40 configurations fine-tuned. Proxy = combined normalized score. indicates proxy-selected top-5 configs.

Configuration	Params	Acc (%)	Lat. (ms)	Proxy
Linear probe	77K	60.0	5.76	–
Structured configurations				
Attn-only r=8	295K	74.6†	6.21	0.45
High-attn low-MLP	958K	76.6	–	0.75
Uniform r=8	1.03M	75.7	6.98	0.44
MLP-only r=16	1.47M	76.5	–	0.39
Increasing rank	1.94M	76.3	6.91	0.08
Decreasing rank	1.94M	76.1	6.92	0.42
Uniform r=16	2.06M	76.5	6.77	**0.82**
Uniform r=32	4.13M	77.3	6.89	0.52
Evolutionary / random search				
rand_2*	1.73M	**79.3**	–	**0.79**
evo_5*	1.51M	77.5	–	0.76
evo_36*	1.40M	76.2	–	**0.82**
rand_20*	1.45M	76.1	–	0.77
evo_99*	1.74M	76.7	–	0.80
evo_38	2.00M	78.1	–	0.42
rand_21	1.58M	77.6	–	0.47

79.3%—a spread of 19.3 points. Even among LoRA configurations, the range is 74.6–79.3% (4.7 points).

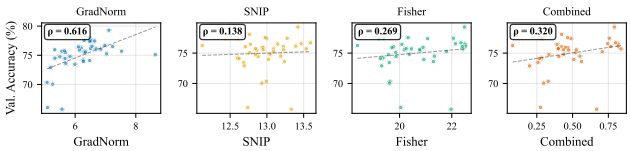

Figure 2. **Proxy–accuracy correlations** on CIFAR-100 (40 configs). GradNorm achieves $\rho = 0.62$ ($p < 0.001$), substantially outperforming other individual proxies.

Non-uniform ranks dominate. The best configuration (rand_2) uses non-uniform per-layer ranks rather than a fixed rank everywhere. Among the top-5 by accuracy, 4 are non-uniform (evolutionary or random search results).

Evolutionary search improves. Seven of the top-10 configurations come from evolutionary or random search rather than handcrafted structured patterns, validating the need for automated topology search.

4.3. Proxy Quality

Figures 2 and 3 show proxy–accuracy correlations. GradNorm achieves $\rho = 0.616$ ($p < 0.001$), substantially stronger than SNIP (0.138), Fisher (0.269), and NegEntropy (0.263). The combined ensemble achieves $\rho = 0.320$

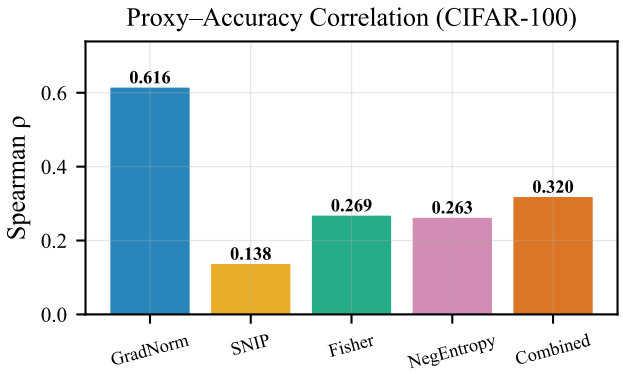

Figure 3. **Per-proxy Spearman ρ comparison.** GradNorm is the dominant signal for adapter topology ranking.

Table 2. **Selection quality.** Accuracy of the best config found in the proxy-selected top-k, compared to oracle (best overall) and random selection baseline.

k	Proxy Best	Oracle	Regret	vs. Random	Hit Rate
1	75.7%	79.3%	3.6	+0.7	0%
3	76.5%	79.3%	2.8	+1.5	0%
5	**79.3%**	79.3%	**0.0**	+4.3	20%
10	**79.3%**	79.3%	**0.0**	+4.3	50%

($p = 0.047$)—interestingly weaker than GradNorm alone, indicating that the weaker proxies dilute the signal when averaged.

This suggests that for adapter topology search specifically, the magnitude of initial gradient flow through adapter parameters (captured by GradNorm) is the most informative zero-cost signal. We hypothesize this is because GradNorm directly measures how much the adapter can learn from a single step, while SNIP and Fisher are more sensitive to the pre-trained backbone's behavior.

4.4. Selection Quality

Table 2 evaluates selection quality. At k=5, the proxy-selected set contains the oracle-best configuration (79.3%), achieving **zero regret**. Even at k=1, the proxy selects a configuration (75.7%) that outperforms random selection by 0.7 points. This demonstrates that despite moderate rank correlation, the proxies are effective at identifying high-performing configurations.

4.5. Cross-Backbone Generalization

Figure 4 and Tab. 3 show cross-backbone results. On ViT-S/16, uniform r=32 achieves 73.6% (vs. 52.6% linear probe), and the configuration rank ordering is consistent with ViT-B/16. The absolute gap (∼4–5 points) is expected given ViT-S's smaller capacity.

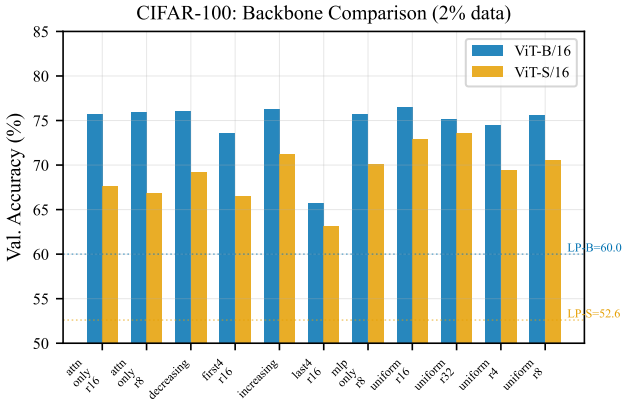

Figure 4. **Backbone comparison** on CIFAR-100 (2% data). ViT-B/16 consistently outperforms ViT-S/16, but rank ordering is preserved.

Table 3. **ViT-S/16 results** on CIFAR-100 (2% data). Rank ordering across configs is consistent with ViT-B/16.

Configuration	ViT-B/16	ViT-S/16
Linear probe	60.0%	52.6%
Uniform r=8	75.7%	70.5%
Uniform r=16	76.5%	72.9%
Uniform r=32	77.3%	73.6%
Increasing rank	76.3%	71.3%
Decreasing rank	76.1%	69.2%

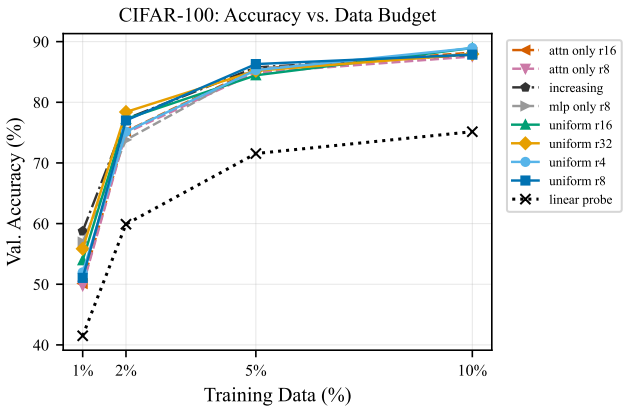

Figure 5. **Data regime sweep** on CIFAR-100. All adapter configs substantially outperform linear probing across data budgets.

4.6. Data Regime Analysis

Figure 5 and Tab. 4 show performance across data budgets from 1% to 10%. The advantage of LoRA over linear probing is consistent (14–19 points) across all data fractions. At 10% data (5,000 samples), the best configuration reaches 89.0% with the smallest rank (uniform r=4), while at 5% the best is uniform r=8 (86.3%). This trend—optimal rank

Table 4. **Best accuracy by data fraction.** LP = linear probe.

Data	Samples	Best Config	Best Acc	LP
1%	500	increasing	58.8%	41.5%
2%	1,000	uniform r=32	78.4%	59.9%
5%	2,500	uniform r=8	86.3%	71.6%
10%	5,000	uniform r=4	89.0%	75.2%

Table 5. **Flowers-102 results (ViT-B/16).** Train on 2,040 images (train+val), evaluate on 6,149 test images. The task is near-saturated: all adapter configs achieve >99%.

Configuration	Params	Acc (%)
Linear probe	78K	98.85
Attn-only r=8	295K	**99.53**
Even-layers r=8	516K	99.51
High-attn low-MLP	958K	99.51
MLP-only r=8	737K	99.50
Decreasing rank	1.94M	99.48
Increasing rank	1.94M	99.45
Middle-4 r=8	344K	99.37

decreasing as data increases—is consistent with reduced overfitting risk at larger data budgets and provides a useful heuristic: use smaller ranks when more data is available.

4.7. Cross-Dataset: Flowers-102

Table 5 shows results on Flowers-102, a fine-grained classification benchmark. The task is near-saturated: even the linear probe achieves 98.85%, and all adapter configurations score between 99.3–99.5%. The best adapter (attn-only r=8) achieves 99.53% with only 295K parameters. The narrow accuracy spread (0.2 points) means proxy ranking is uninformative on this dataset ($\rho \approx 0$), which is expected: when all configurations perform near-identically, there is no signal for proxies to detect. This confirms that AdapterNAS is most valuable when the adapter topology choice has meaningful impact on accuracy, as in the CIFAR-100 low-data regime.

4.8. Pareto Analysis

Figure 6 shows the Pareto front. The initial rise is steep—going from 77K (linear probe) to 295K (attn-only r=8) yields a 14.6 point accuracy gain. Beyond ∼1M parameters, gains become incremental, with the best config at 1.73M. The Pareto-optimal set includes both structured (uniform) and search-discovered (evolutionary) configurations.

4.9. Evolutionary Search Convergence

Figure 7 shows how the evolutionary search improves the proxy score distribution over 6 generations. The best proxy score increases from the initial population (structured + random), validating that the evolutionary procedure discovers configurations that the handcrafted search space misses.

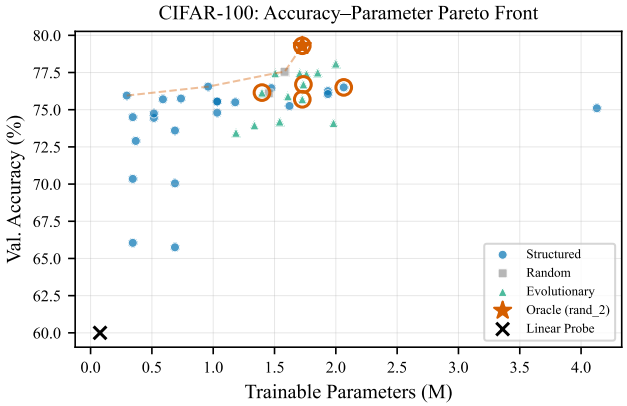

Figure 6. **Accuracy vs. trainable parameters** on CIFAR-100. Red circles highlight proxy-selected top-5. The oracle-best (rand_2) lies in the proxy top-5, achieving 79.3% with only 1.73M adapter parameters.

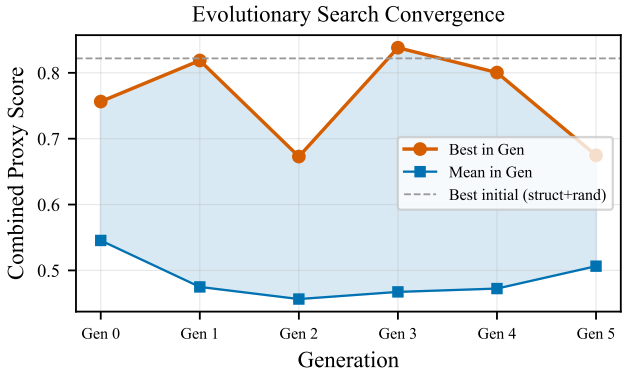

Figure 7. **Evolutionary search convergence.** Best and mean proxy scores by generation. The search improves over the initial population within 2–3 generations.

Table 6. **Search cost comparison.**

Method	Configs	Time	Hardware
AdapterNAS (proxy scoring)	175	15 min	1× A10G
Grid search (5 ep. each)	175	∼15 hrs	1× A10G
AdaLoRA [22]	1 (adaptive)	∼1 hr	1× GPU

4.10. Search Cost

AdapterNAS scores 175 configurations in 15 minutes using only single-sample forward-backward passes, achieving a **60× speedup** over exhaustive fine-tuning. Importantly, the search itself is training-free: the 15 minutes consists entirely of proxy computation, with no gradient steps for optimization.

5. Discussion

GradNorm as the primary signal. The dominance of GradNorm ($\rho = 0.62$) over other proxies is notable. We conjecture this is because adapter parameters are initialized near zero (via the standard LoRA initialization $A \sim \mathcal{N}(0, \sigma^2)$, $B = 0$), and GradNorm directly measures the magnitude of the training signal flowing into these new parameters. SNIP and Fisher, designed for pruning at initialization in randomly initialized networks, are less well-suited to the adapter setting where the backbone is already well-trained.

Combined proxy is weaker than GradNorm. Counter-intuitively, the equal-weight ensemble ($\rho = 0.32$) underperforms GradNorm alone ($\rho = 0.62$). This suggests that for adapter-specific search, a weighted or learned combination—or simply using GradNorm alone—may be preferable.

Implications for LoRA practice. (1) Adapt *all* modules (QKV and MLP), not just attention. MLP-only r=16 (76.5%) matches uniform r=16 (76.5%), and MLP-only r=8 at 737K params outperforms attention-only r=16 at 590K. (2) Non-uniform, per-layer rank allocation can outperform uniform strategies by 2–3 points. (3) Early layers benefit more: first-4 r=16 substantially outperforms last-4 r=16 on CIFAR-100.

Clarification on "training-free." AdapterNAS is training-free in the *search* phase: candidate configurations are ranked using zero-cost proxies without any fine-tuning. The selected configuration is then fine-tuned normally. This is analogous to training-free NAS [1], where the search is training-free but the final architecture is trained.

6. Conclusion

We introduced AdapterNAS, a training-free NAS framework for adapter topology search in foundation models. By combining zero-cost proxies with evolutionary optimization, AdapterNAS searches 175 LoRA configurations in 15 minutes and identifies top-performing topologies with zero regret at k=5. Our experiments on ViT-B/16 and ViT-S/16 across CIFAR-100 (up to 89.0% at 10% data), Flowers-102 (>99% with only 295K adapter parameters), and four data regimes demonstrate that adapter topology is a critical design choice with 19+ point accuracy impact, and that GradNorm provides a strong ($\rho = 0.62$) proxy signal for guiding this choice without training.

Limitations and Future Work. We evaluate on ViT with two vision datasets. On Flowers-102, the task is near-saturated and adapter topology has minimal impact, limiting the proxy's utility—further evaluation on more challenging fine-grained benchmarks (*e.g.*, iNaturalist, CUB-200) is warranted. Extending to language models, larger ViTs, and multi-modal architectures is important. The strong performance of GradNorm suggests developing adapter-specific proxies is a promising direction. Weighted proxy combinations and learned proxy selection may further improve selection quality.

References

[1] Mohamed S. Abdelfattah, Abhinav Mehrotra, Lukasz Dudziak, and Nicholas D. Lane. Zero-cost proxies for lightweight NAS. In *ICLR*, 2021. 1, 2, 6

[2] Shoufa Chen, Chongjian Ge, Zhan Tong, Jiangliu Wang, Yibing Song, Jue Wang, and Ping Luo. AdaptFormer: Adapting vision transformers for scalable visual recognition. In *NeurIPS*, 2022. 1

[3] Wuyang Chen, Xinyu Gong, and Zhangyang Wang. TE-NAS: Searching for better training-free neural architecture evaluation metrics. In *ICLR*, 2021. 2

[4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *NeurIPS*, 2023. 1

[5] Xuanyi Dong and Yi Yang. NAS-Bench-201: Extending the scope of reproducible neural architecture search. In *ICLR*, 2020. 2

[6] Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. Towards a unified view of parameter-efficient transfer learning. In *ICLR*, 2022. 1

[7] Neil Houlsby, Andrei Gimenez-Abalos, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *ICML*, 2019. 1

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022. 1

[9] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *ECCV*, 2022. 1

[10] Dawid J. Kopiczko, Tijmen Blankevoort, and Yuki M. Asano. VeRA: Vector-based random matrix adaptation. In *ICLR*, 2024. 1

[11] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. 2

[12] Namhoon Lee, Thalaiyasingam Ajanthan, and Philip Torr. SNIP: Single-shot network pruning based on connection sensitivity. In *ICLR*, 2019. 2

[13] Dongze Lian, Daquan Zhou, Jiashi Feng, and Xinchao Wang. Scaling & shifting your features: A new baseline for efficient model tuning. In *NeurIPS*, 2022. 1

[14] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. In *ICML*, 2024. 1

[15] Joseph Mellor, Jack Turner, Amos Sherrel, and Elliot Sherrel. Neural architecture search without training. In *ICML*, 2021. 1, 2

[16] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *Indian Conference on Computer Vision, Graphics and Image Processing*, 2008. 2

[17] Hidenori Tanaka, Daniel Kunin, Daniel L.K. Yamins, and Surya Ganguli. Pruning neural networks without any data by iteratively conserving synaptic flow. In *NeurIPS*, 2020. 2

[18] Mojtaba Valipour, Mehdi Rezagholizadeh, Ivan Kobyzev, and Ali Ghodsi. DyLoRA: Parameter efficient tuning of pre-trained models using dynamic search-free low-rank adaptation. In *EACL*, 2023. 1

[19] Chaoqi Wang, Guodong Zhang, and Roger Grosse. Picking winning tickets before training by preserving gradient flow. In *ICLR*, 2020. 2

[20] Ross Wightman. Pytorch image models. *GitHub repository*, 2019. 2

[21] Chris Ying, Aaron Klein, Eric Christiansen, Esteban Real, Kevin Murphy, and Frank Hutter. NAS-Bench-101: Towards reproducible neural architecture search. In *ICML*, 2019. 2

[22] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In *ICLR*, 2023. 1, 5

[23] Han Zhou, Xingchen Lu, Yusheng Shen, Tao Zhao, and Jian Zeng. AutoPEFT: Automatic configuration search for parameter-efficient fine-tuning. *TACL*, 2024. 1